College for Professional Studies
3333 Regis Boulevard
Denver, CO 80221

Regis University
College for Professional Studies
School of Computer & Information Sciences

FACILITATOR'S SYLLABUS

NOTE to Students: This syllabus is the SAME as the syllabus posted on the Regis website, with your facilitator's contact information and personal policies added. All additions are in GREEN, like this comment.

COURSE TITLE: Control Structures

COURSE NUMBER: CS 361

INSTRUCTOR INFORMATION:

Name:	Greg Shubert
Class Location:	Online
Telephone Number:	970.402.3863 Call between 9 am and 9 pm, Mountain Daylight Time.
Email:	gshubert@regis.edu
Facilitator Expectations:	If a posting on the forum has not been resolved within 48 hours, I will intervene. I will respond to personal questions sent to me via email within 48 hours. Please use a meaningful subject line; for example "CS 361: grade for hw 1". I will post and email graded homework within 96 hours after it is due (for example, end of the day Thursday for assignments due Sunday).

COURSE DESCRIPTION: CS 361. CONTROL STRUCTURES (3). Discusses basic concepts of computer organization. Develops a precise and logical methodology in reducing complex and unformatted data to algorithmic format. Introduces the concepts and methodologies of structured programming and design. Demonstrates the uses, abuses, and best practices of control structures.

Expanded Description:
CS361 is an introductory computer-programming course in which the control structure aspects of programming languages are taught using the C++ programming language. The control structures

covered in this course form the basis of virtually all structured and object-oriented programming languages. However, the CS361 course will be taught using structured C++, and object-oriented programming will not be covered.

In this course, the student is introduced to console application programs that use constant and variables of primitive data types, input/output processing, sequential, selection, and iterative control structures, and functions for modular programming. The student is also introduced to the fundamentals of software engineering methodology, including algorithm creation, top down design, and structured programming.

PREREQUISITE COURSES: None

Although there are no ***official*** pre-requisites for this course, undergraduate students majoring in CS/CIS/CN who are new to programming will be expected to take **CS208 (Computer Science Fundamentals)** before CS361, and should:

Know what software is and have used some form of commercial software
Be familiar with basic hardware components of a computer (*CS208*)
Understand the difference between low level and high level programming languages (*CS208*)
Have been introduced to the basic structure of C++ programs (*CS208*)
Be able to compile and run a simple C++ program (*CS208*)
Have access to and experience using the Internet (e-mail and a web browser).
Have sufficient time to devote ***at least*** 20 hours per week to this course.

CS361 is challenging and covers a substantial amount of material at a rapid pace. Ultimately, programming is a skill that requires the ability to put designs into practice. The only way to succeed is to practice this skill.

Note that if you are a new programmer, you may require a **significant** amount of time each week to complete the programming assignments. If you are not willing or able to spend the necessary time, please reconsider whether this is the correct time to attend this class.

COURSE OUTCOMES:

Upon completion of this course, learners should be able to:

1. Explain the importance of algorithms in the problem-solving process and create algorithms for solving simple problems.
2. Use pseudo-code, flowcharting, or a high-level programming language to implement algorithms for solving small business and scientific problems.
3. Discuss the representation and usage of primitive data types.
4. Design, implement, test, and debug programs that use each of the following fundamental programming constructs: basic computation, simple input/output, standard sequential, conditional, and iterative control structures, and functions. Choose appropriate constructs for each programming task.

5. Apply structured (functional) decomposition to break a program into smaller pieces, and to modify and expand programs.
6. Compare and contrast parameter passing by value and by reference, and describe the mechanics of each.
7. Describe the concept of recursion and explain its appropriateness to certain problems. Compare and contrast iterative and recursive solutions.
8. Identify the base and general cases of a recursively defined problem. Implement, test, and debug a simple recursive function.
9. Describe strategies that are useful in debugging.
10. Analyze and explain the behavior of programs that use the fundamental programming constructs introduced in this course. Use the scientific method and other strategies to identify program deficiencies.

COURSE MATERIALS

Required Texts:
Malik, D.S. (2013). *C++ Programming: From Problem Analysis to Program Design, 6th edition.* Boston, MA: Cengage Learning; ISBN-13: 978-1-133-62638-1.

Textbook Note:
Because this book is used in multiple required courses, we do not recommend the "book rental" option. The same textbook will also be used in CS362 -- so save it!

Technology Tools (i.e. Required Hardware/Software):

1. A PC-compatible computer system running Windows. **(But don't panic if you have a Mac or Linux system.)**

2. A C++ Compiler -- the preferred compiler is the free **Dev-C++** compiler

The newest Windows 8-compliant version of Orwell Dev-C++, can be downloaded from:
http://sourceforge.net/projects/orwelldevcpp/files/Setup%20Releases/

Download the most recent version:
Dev-Cpp x.x.x MinGW x.x.x Setup.exe (32-bit version)
OR
Dev-Cpp x.x.x TDM-GCC x64 x.x.x Setup.exe (64-bit version)

WARNING: If you had the older version loaded on your computer from an earlier class, such as the Bloodshed Dev-C++ compiler, ***you must uninstall it*** before installing the newer version.

Other ANSI standard C++ compilers can be used, but the facilitator will be most familiar with the Dev-C++ compiler listed above, and will most likely use the Dev-C++ compiler to grade your programs. If a student chooses to use another compiler, it is the student's

responsibility to obtain any help needed to use the software, and to insure that all assigned programs compile and run correctly on the Dev-C++ compiler.

I have access to a Dev-C++ compiler but generally use a similar compiler (g++ or Code::Blocks) on Mac OS X or Linux operating systems. I try to support all these compilers, although I may not be familiar with a different IDE (integrated development environment).

As with most of Regis learning activities, using various software applications to accomplish assignments requires students to exercise a great deal of responsibility for learning how to successfully operate the software applications. This class assumes that you have a working knowledge of a C++ compiler (from CS208 or prior programming experience). There are, however, many quality books on the market that support novice users if you need them.

PRE-ASSIGNMENT: (classroom only; good to read for online students)

Complete the following **two** tasks before the first night of class:

1) Reading Assignment: Read **pages 1 to 153**, and **Appendix E pages 1278 and 1279** in the textbook. Be prepared to ask questions on unclear areas and to respond to questions about information in the assigned reading.

2) Written Assignment: An algorithm is defined as:

> "a precise description of a method for solving a particular problem using operations or actions from a well-understood repertoire. Algorithms are everywhere. When we change the tires on a car, mow the lawn, vacuum a rug, or follow directions to a restaurant, we use an algorithm. A person can solve problems by means of algorithms expressed in English, provided the person understands English. And a computer can solve problems by means of algorithms expressed in a programming language, provided that the computer 'understands' that language."
> [From John Shore, *The Sachertorte Algorithm*, 1985]

Choose any everyday action, such as those listed above, and **write** a specific, clear, and precise **algorithm** for it, in English. You may number the different steps, if needed.

RELATED STANDARDS

Program Outcomes	
	This program enables students to achieve the following program outcomes by the time of graduation:
	An ability to apply knowledge of computing and mathematics appropriate to the discipline
	An ability to analyze a problem, and identify and define the computing requirements appropriate to its solution
	An ability to design, implement, and evaluate a computer-based system, process, component, or program to meet desired needs
	An ability to use current techniques, skills, and tools necessary for computing practice
	An ability to apply mathematical foundations, algorithmic principles, and computer science theory in the modeling and design of computer-based systems in a way that demonstrates comprehension of the tradeoffs involved in design choices
	An ability to apply design and development principles in the construction of software systems of varying complexity
Regis Nine Educational Outcomes	
	Content Knowledge in Discipline
	Diverse Cultures, Perspectives, and Belief Systems
	Arts, Sciences, and Humanities
	Critical Thinking
	Communication
	Technology
	Ethical and Social Responsibility
	Leadership in Service to Others
	Lifelong Learning
Jesuit Themes - These are themes that are embedded as part of our interactions and instruction at Regis, but are not written in as activities	
	Magis, Finding God in All Things, Unity of Hearts and Mind, Contemplatives in Action, Care of the Whole Person, Men and Women for Others

COURSE ASSIGNMENTS AND ACTIVITIES:

Week	Learning Topics	Readings (textbook 6th ed*)	Graded Assignments and Associated Points
1	LT 1 – Algorithmic Problem-Solving and Programming Strategies LT 2 – Syntax and Semantics of a High-Level Programming Language LT 3 – Types, Variables, and Assignment LT 4 – Input and Output LT 5 – Arithmetic Expressions and Operators	**Chapter 1**, all **Chapter 2**, all **Chapter 3**, pp. 121 – 153 (Input/Output) **Appendix E**, pp. 1278 – 1279 (Header File Names) ***Optional Exercises*** (answers in back of text): Odd exercises 1 - 33 on pp. 105 – 113, and odd exercises 1 - 9 on pp. 171 - 173	First Week Assignment and Participation (for all weeks) 10% of total All Programing Assns 52% of total Prog Assn 1 4% of 52%
2	LT 6 – part I: Sequential Control LT 7 – part I: void Functions and Parameter Passing by Value	**Chapter 6**, pp. 336 – 340 (pre-defined functions), pp. 364-365, 370-371 (void functions/value parameters) **Appendix F**, pp. 1281 – top of 1287 (header files) ***Optional Exercise*** (answer in back of text): Exercise 3 on pp. 422	Prog Assn 2 8% of 52%
3	LT 5 – Boolean Expressions and Operators LT 6 – part II: Conditional Control Structures LT 7 – part II: value-returning Functions	**Chapter 4**, pp. 184 – top of 204 (if/else) **Chapter 6**, pp. 341 – top of 348 (value-returning functions) ***Optional Exercises*** (answers in back of text): Odd exercises #1 - 13 on pp. 240 – 243, and exercise #1 on p. 422	Prog Assn 3 15% of 52%
4	LT 6 – part II: Conditional Control Structures	**Chapter 4**, pp. 204 – top of 214 (nested and extended ifs), bottom of 217 – top of 231 (conditional operator and switch), and bottom of 233 – 239 (example program and review) ***Optional Exercises*** (answers in back of text): Odd exercises 15 - 31 on pp. 243 - 250	Prog Assn 4 15% of 52% Midterm Exam 19% of total
5	LT 6 – part III: Iterative Control Structures	**Chapter 5**, pp. 260 – top 305 (loops) **Chapter 6**, pp. 348 – 363 (prototypes, examples, and review) ***Optional Exercises*** (answers in back of text): Odd exercises 1 - 43 on pp. 315 – 327, odd exercises 5 - 19 on pp. 423 - 428	Prog Assn 5 14% of 52%

6	LT 6 – part III: Iterative Control Structures (nested) LT 7 – part III: Parameter Passing by Reference	**Chapter 5**, pp. 305 – 315 (nested loops, debugging, review) **Chapter 7**, pp. 364 – top of 386 (value/reference parameters) ***Optional Exercises*** (answers in back of text): Odd exercises 21 - 35 on pp. 429 - 435	Prog Assn 6 22% of 52%
7	LT 7 – part II: Functions and Parameter Passing by Reference LT 8 – Recursion	**Chapter 7**, pp. bottom 386 – 398 (scope, types, debugging), pp. 401 – top of 408 (default parameters and example), pp. btm 418 – 421 (review) **Chapter 15**, pp. 986 – top of 989 and pp. 993 – top 1009 (recursion) ***Optional Exercises*** (answers in back of text): Exercise #37 on p. 435, and odd exercises 1 - 13 on pp. 1009 - 1011	
8		End of Chapter Quick Reviews for Ch 1-7 & 15	Prog Assn 7 22% of 52% Final Exam 19% of total

****Alternate Edition Reading Assignments:*** Alternate reading assignments for the previous edition of the textbook (5th edition) are located in **Appendix B** of this syllabus.

Programming Assignments

There will be 7 programming assignments. Some of the programs are easier than other programs, so each programming assignment is worth a different percentage, as shown in the grids above and below.

Assignment	Value (percent of overall course grade)
Programming Assignments (7 assignments) Program 1 is worth 4% of the 52% Program 2 is worth 8% of the 52% Program 3 is worth 15% of the 52% Program 4 is worth 15% of the 52% Program 5 is worth 14% of the 52% Program 6 is worth 22% of the 52% Program 7 is worth 22% of the 52%	52 %
Midterm Exam	19 %
Final Exam	19 %
Participation and First Night Assn	10 %
Course Total	100 %

Each programming assignment will involve writing two programs, using the concepts discussed in the book and class, and may also include submission of pseudocode, flowcharts, test data and/or test plans. Programming assignments will be graded according to the following criteria:

1. Source Code
   - The code is easy to read, i.e. formatted nicely (spaces/blank lines used).
   - The code follows the coding standards provided.
   - The code includes in-line documentation.
2. Coding Style
   - The program is modular.
   - The C++ programming language is used correctly (structured, non-OO).
3. Functionality
   - There are no compilation/logic/formatting errors, etc.
   - The program solves assigned problem
4. Pseudocode/FlowChart (if required)
   – Algorithm is correctly depicted
5. Test Plan (if required)
   – Includes both test input and expected output for each test case
   – Includes all cases necessary to validate the program code

Exams

There will be a midterm exam and a final exam. Exam questions will be cumulative, taken from reading assignments and class presentations. The questions will include: syntax, evaluating expressions, tracing code fragments, reading and writing code.

Participation

Class participation/effort is important because we can all learn from each other. Your participation points can make a difference in the final grade. Participation means:

1. Being present in the forum every week.

2. Regularly checks forum and posts all required items by the deadlines.

3. Replies to other students in forum discussions.

4. Timely, regular participation is vital. A student who does not participate during a given week will lose participation points that week. These points cannot be "made up", for example, by posting comments for Week 4 during Week 5.

Weekly Discussion points distribution

9 points per correct Discussion Question/Test Case response each week
6 points for ***at least one*** question or comment about the week's topics
(each week in weeks 1 – 7)
6 points per Weekly Feedback in weeks 1 – 7
8 points for Course Feedback in week 8

Partial points may be given for late, incomplete, or partially incorrect posts.
No points will be given for items posted ***more than one week*** past the due date.

Week 1 there are 30 possible points
Week 2 there are 21 possible points
Week 3 there are 39 possible points
Week 4 there are 21 possible points
Week 5 there are 30 possible points
Week 6 there are 21 possible points
Week 7 there are 30 possible points
Week 8 there are 8 possible points

Total forum points available: **200 points** = 10% of total course grade

GRADING SCALE:

Letter Grade	*Percentage*	*Grade Point*
A	93 to 100	4.00
A–	90 to less than 93	3.67
B+	87 to less than 90	3.33
B	83 to less than 87	3.00
B–	80 to less than 83	2.67
C+	77 to less than 80	2.33
C	73 to less than 77	2.00
C–	70 to less than 73	1.67
D+	67 to less than 70	1.33
D	63 to less than 67	1.00
D–	60 to less than 63	.67
F	Less than 60	0

COLLEGE FOR PROFESSIONAL STUDIES POLICIES (LINKS):

You will be asked to enter a valid Regis University UserID and password to access the following policies.

Academic Integrity
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/Academic%20Integrity%20Policy%20and%20Board%20-%20December%2020,%202010.doc

Plagiarism includes submitting code obtained from any person or from any internet web source. All work must be your own.

Other forms of academic dishonesty may include but are not limited to: misuse or abuse of the Regis web site and course materials (such as copying them to other web sites); or publishing your programs or answers to questions online so they are publicly accessible.

In CS361, in cases of suspected cheating or plagiarism,the instructor will discuss the matter with the student(s) involved. The instructor reserves the right to question any student orally or in writing about any assignment, and to use the evaluation of the student's understanding of the assignment and of the submitted solution as evidence of cheating.

Attendance Participation
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Attendance%20Participation.pdf

Confidential Proprietary Information Policy
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Confidential%20Proprietary%20Information%20Policy.pdf

Dayton Memorial Library
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Dayton%20Memorial%20Library.pdf

Diversity
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Diversity%20at%20RU.pdf

Equal Access and Disability Services
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Equal%20Access_Disability%20Services.pdf

Human Subjects Review (IRB)
http://www.regis.edu/Academics/Academic-Research-and-Grants/Regis%20Review%20Boards.aspx#.UUtE8WdYkdU

Inclement Weather and Class Cancellation
In the event of inclement weather (i.e., blizzard), students should call the University Weather line at 303-458-1818 for closure information.

Late or Incomplete Course Procedures
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Late%20Assignments_Assignment%20Revisions%20Policy%20CPS.pdf

Late Assignment Policy for CS361

Late programming assignments will be graded and then 2% will be deducted for each day the assignment is late, ***up to 5 days late***. No programming assignment will be accepted more than ***5 days*** after the official due date. Therefore, any programming assignment turned in more than ***5 days*** late will be given a grade of **zero**, and no feedback will be given.

ONLINE: Late discussion posts will have 10% deducted for each day the post is late, up to one week late. Any discussion assignment posted more than one week late will be given a grade of **zero**.

Learner Conduct
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Learner%20Conduct.pdf

Writing Assistance
https://in2.regis.edu/sites/spsdean/CPS%20Approved%20Policies%20and%20Procedures/CPS%20Syllabus%20Policies/Writing%20Assistance.pdf

OTHER INFORMATION:

NOTE TO LEARNERS: On occasion, the course facilitator may, at his or her discretion, alter the Learning Activities shown in this Syllabus. The alteration of Learning Activities may not, in any way, change the Learner Outcomes or the grading scale for this course as contained in this syllabus. Examples of circumstances that could justify alterations in Learning Activities could include number of learners in the course; compelling current events; special facilitator experience or expertise; or unanticipated disruptions to class session schedule.

Additional Policies

The Forum is the place for any questions relevant to the course material. I've discovered that most email questions are of general interest and the answers are helpful to the class in general. If you send me such an email, I will ask you to post it to the Forum.

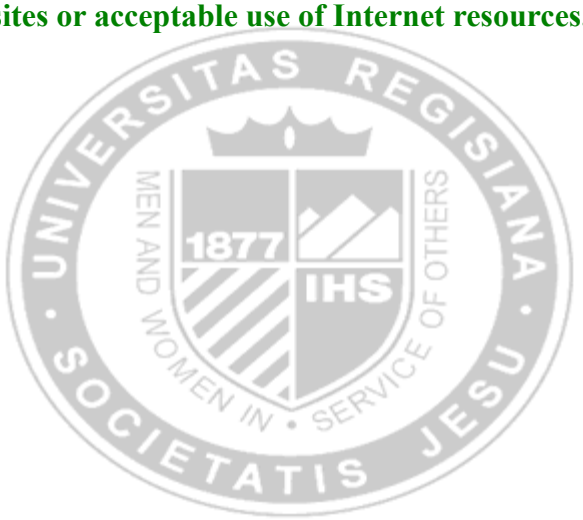
Try to keep email and Forum postings friendly and courteous. It is easy to be misunderstood in such text-only forms. Please contact me by phone or email if you have something that cannot be posted on the Forum.

Learning programming is like learning a new language. You are not just learning new facts. You are learning new ways of thinking. It is okay to ask questions as soon as you think of them. Clear up uncertainties and get answers to questions soon, and it is easier to build upon that new knowledge and understanding.

I encourage you to ask as many questions on the Forum as you need to understand the material. Also, I invite you to respond to each others' questions and not wait for the facilitator. Your learning is enhanced by trying to explain a topic to someone, or by asking followup questions. But one learns nothing if simply given an answer. Hence, post no solutions -- just hints. Please contact me if you have a question about acceptable Forum postings.

If you have trouble with a programming assignment, post only the relevant small portion of your code -- not the whole program -- with your questions. Try to be specific in your questions. "I expected this code to do X but it did Y instead" is good. "Help!" is not.

While the Internet is full of information, much is neither useful nor appropriate. It can be hard for beginners to distinguish good sites from not-so-good. Please contact me if you have a question about useful sites or acceptable use of Internet resources.



APPENDIX B

CS361 Alternate Readings for 5th edition of Textbook

Week	Learning Topics	Readings (textbook 5th ed)	Optional Textbook Exercises (answers in back of book)
1	LT 1 – Algorithmic Problem-Solving and Programming Strategies LT 2 – Syntax and Semantics of a High-Level Programming Language LT 3 – Types, Variables, and Assignment LT 4 – Input and Output LT 5 – Arithmetic Expressions and Operators	Chapter 1, all Chapter 2, all Chapter 3, pp. 117 – 148 Appendix E, pp. 1244 – 1245	Odd exercises #1 - 33 on pp. 100 – 108 Odd exercises #1 - 7 on pp. 166 - 168
2	LT 6 – part I: Sequential Control LT 7 – part I: Functions and Parameter Passing by Value	Chapter 6, pp. 320 – 323 Chapter 7, pp. 362-363, 367-368 Appendix F, pp. 1247 – top of 1253	Exercise #3 on pp. 351
3	LT 5 – Boolean Expressions and Operators LT 6 – part II: Conditional Control Structures LT 7 – part II: value-returning Functions	Chapter 4, pp. 176 – 195 Chapter 6, pp. 324 – top of 331	Odd exercises #1 - 7 on pp. 232 – 235 Exercise #1 on p. 350
4	LT 6 – part II: Conditional Control Structures	Chapter 4, pp. 195 – 206, 209 – top of 223, and 225 – 232	Odd exercises #9 - 23 on pp. 235 - 240
5	LT 6 – part III: Iterative Control Structures	Chapter 5, pp. 248 – top of 291 Chapter 6, pp. 331 – 350	Odd exercises #1 - 41 on pp. 301 - 311 Exercises #5 - 15 on pp. 351 - 355
6	LT 6 – part III: Iterative Control Structures (nested) LT 7 – part III: Parameter Passing by Reference	Chapter 5, pp. 291 – 301 Chapter 7, pp. 362 – top of 382	Odd exercises #1 - 17 on pp. 416 - 423
7	LT 7 – part II: Functions and Parameter Passing by Reference LT 8 – Recursion	Chapter 7, pp. bottom 382 – 394 pp. 397 – top of 404 pp. btm 414 – top 416 Chapter 17, pp. 992 – top of 995 pp. btm 999 – top 1015	
8		All End of Chapter Quick Reviews	


UNIVERSITAS REGISIANA
SOCIETATIS JESU
1877
IHS
MEN AND WOMEN IN SERVICE OF OTHERS